Git and GitHub

Understanding Version Control and Collaborative Development

Umama Ume Amen | BSc Artificial Intelligence, UET Lahore

Figure 1: The Git logo (left) and the GitHub logo (right)

Introduction

Anyone who starts writing code sooner or later runs into the same **problem: files change constantly, mistakes creep in, and it becomes hard to remember what was changed and why**. This is exactly the problem that **Git** was built to solve. This report looks at Git and GitHub together, since students often use the two names interchangeably even though they are not the same thing. The goal here is to explain what each one actually does, how they work together, and why they have become close to standard practice for anyone writing code today, whether alone or in a team.

What is Git?

Git is a version control system, which is really just a program that **keeps track of every change made to a set of files over time**. It runs locally on a computer and does not need an internet connection to work. Every time a set of changes is saved (this is called a commit), Git stores a snapshot that can be revisited later. This means that if something breaks, it is possible to go back to an earlier working version instead of starting over, originally to help manage the source code of the Linux kernel, and it has since become the most widely used version control tool in software development.

What is GitHub?

GitHub is a website and cloud platform that **hosts Git repositories online**. It takes the local version control that Git already provides and adds a remote, shareable copy that any team member can access from anywhere. Beyond just storage, GitHub adds a whole layer of collaboration tools on top of Git, such as pull requests, issue tracking, and automated workflows, which are not part of Git itself.

Git vs GitHub: What is the Difference?

The easiest way to remember the difference is this**: Git is the tool, and GitHub is a place to store and share the results of that tool.** Git is a piece of software that a developer installs and runs on their own machine to track changes locally. GitHub, on the other hand, is a cloud-based hosting service built around Git repositories, and it is only one of several platforms that do this job (GitLab and Bitbucket are two others). In short, a person can use Git without ever touching GitHub, but GitHub would not exist without Git as its foundation.

Overview

Why Git is Used

Git is used because manual version tracking, such as saving files with names like project-final-v2-reallyfinal.docx, simply does not scale. Git automates this by recording **exactly what changed, when, and by whom, and it allows multiple people to work on the same project without overwriting each other's work**. It also allows developers to experiment freely using branches, since any change that does not work out can simply be discarded without affecting the main project.

Why GitHub is Used

GitHub is used **because it turns a local Git repository into something a whole team**, or the whole world, can access. It gives every project a public or private home, along with tools for reviewing code before it is merged, discussing bugs, and even automatically testing code whenever changes are pushed. For students and early developers, GitHub also acts as a **public portfolio**, since employers frequently look at a candidate's GitHub profile to see real project work.

Real-World Applications

- Software teams at companies of every size use Git and GitHub to coordinate development across multiple developers and time zones.
- Open-source projects such as Linux, VS Code, and TensorFlow rely on GitHub to accept contributions from thousands of independent developers.
- University students and freelancers, including in coursework and Fiverr-style project delivery, use GitHub to store and showcase their work.
- Non-code projects, such as documentation, research notes, or configuration files, are also frequently version-controlled with Git.

Features of Git

- **Version Control:** Every change is recorded as a snapshot, so the complete history of a project is always available and nothing is ever truly lost.
- **Branching:** A new, isolated line of development can be created at any time, allowing a feature or fix to be built without disturbing the main codebase.
- **Merging:** Once work on a branch is complete and tested, it can be combined back into the main branch, bringing the two histories together.
- **Commit History:** A readable log of every commit shows exactly what changed, who made the change, and when it happened.
- **Rollback:** If a recent change causes problems, Git makes it possible to revert the project back to any earlier, working commit.
- **Collaboration:** Multiple developers can work on the same files at the same time, and Git handles combining their changes intelligently.

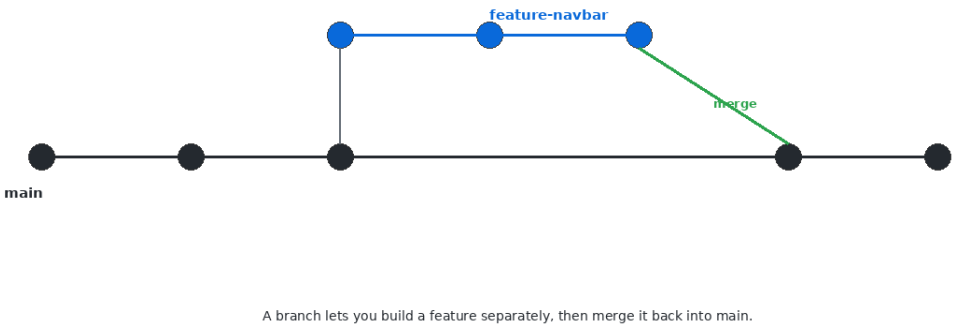

Figure 2: A simplified view of how a feature branch is created from main and later merged back in.

Features of GitHub

- **Remote Repositories:** Every project gets a cloud-hosted copy that acts as the shared source of truth for the whole team.
- **Pull Requests:** A structured way to propose merging one branch into another, complete with a space for discussion and code review before anything is finalized.
- **Issues:** A built-in tracker for bugs, feature requests, and tasks, so nothing gets lost in a chat conversation.
- **Actions (CI/CD):** Automated workflows that can run tests, build the project, or deploy it automatically whenever new code is pushed.
- **Collaboration Tools:** Comments, mentions, and team permissions make it possible for many contributors to work on one project without stepping on each other's toes.
- **Repository Management:** Settings for access control, branch protection rules, and project boards help keep larger projects organized.

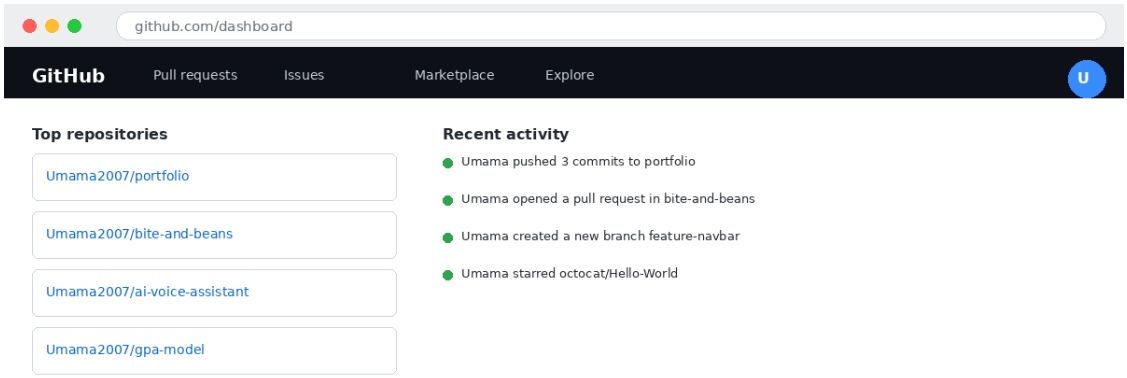

Figure 3: A GitHub dashboard, showing a user's repositories and recent activity feed.

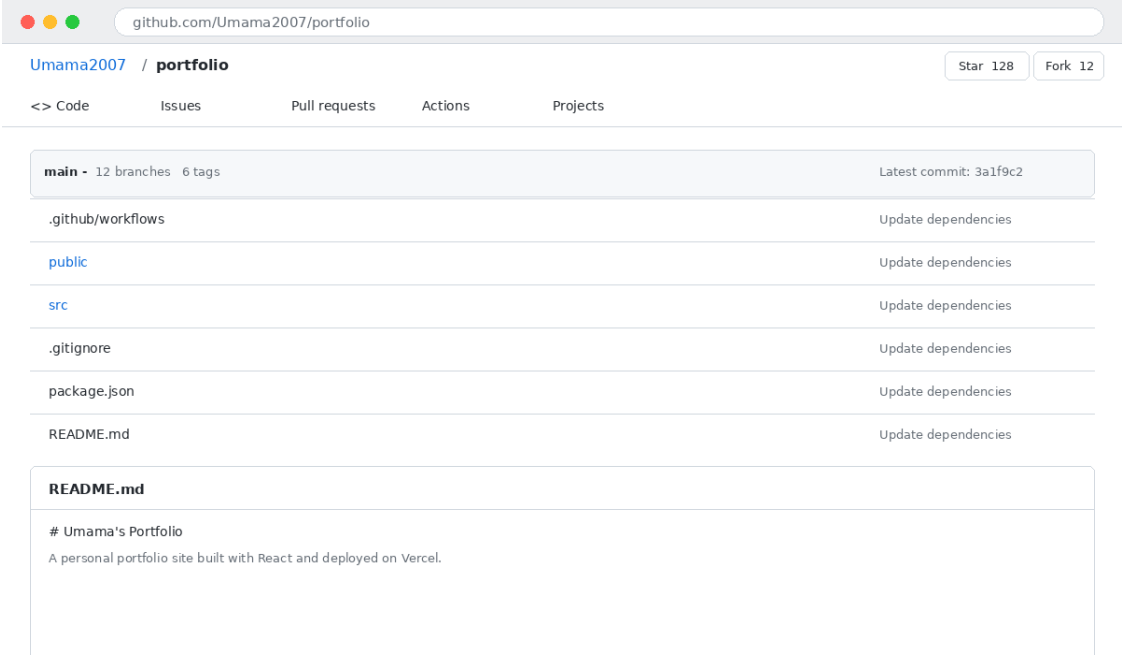

Figure 4: A typical GitHub repository page, listing project files and the README.

Installation

Installing Git on Windows

- Go to git-scm.com and download the latest Windows installer.
- Run the downloaded .exe file to launch the Git Setup wizard.
- Keep the default options selected on each screen unless there is a specific reason to change them (the defaults work well for most beginners).
- Click Install, then Finish once the installation completes.
- Open Git Bash from the Start menu to confirm the installation by typing git --version.

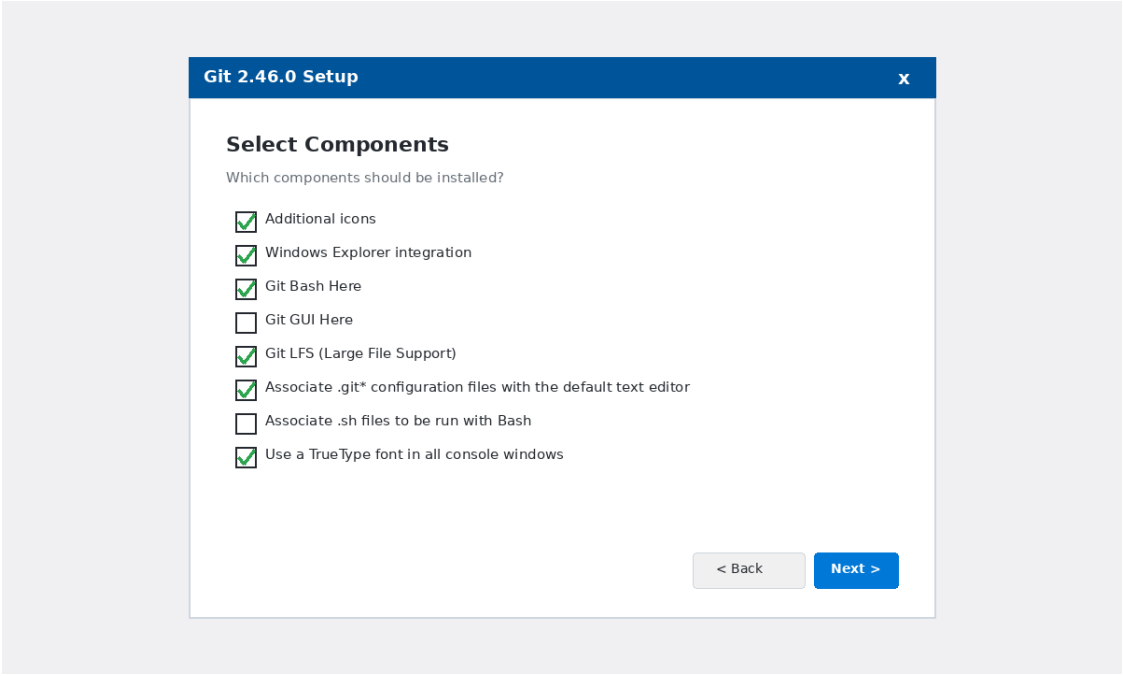

Figure 5: The Git Setup wizard on Windows, showing the component selection screen.

Basic Setup (git config)

Before making any commits, Git needs to know who is making them. This is a one-time setup done from Git Bash or any terminal:

```
git config --global user.name "Umama Ume Amen"
git config --global user.email "umamasajid25@gmail.com"
```

These two settings are attached to every commit made afterwards, which is how GitHub knows which contributor made which change.

Getting Started

Initializing a Repository

Any existing project folder can be turned into a Git repository by running git init inside it. This creates a hidden .git folder that Git uses to store the entire history of the project from that point onward.

Connecting Git to GitHub

Once a project has a GitHub repository created for it, the local Git repository can be linked to it using a remote URL. From then on, git push and git pull are used to send and receive changes between the local machine and GitHub.

Creating a Repository on GitHub

- Log in to GitHub and click the + icon, then New repository.
- Give the repository a name and an optional description.
- Choose whether it should be public or private.
- Optionally add a README file, then click Create repository.

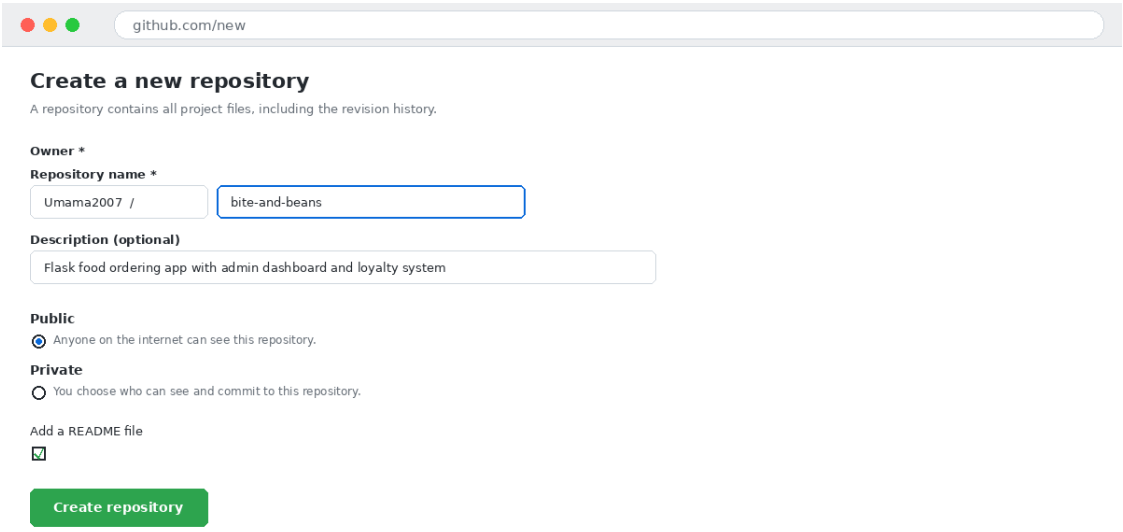

Figure 6: The "Create a new repository" form on GitHub.

Cloning a Repository

To get a copy of an existing GitHub repository onto a local machine, git clone is used along with the repository's URL. This downloads all the files as well as the full commit history, making it possible to continue work exactly where it left off.

Basic Git Commands

The table below summarizes the commands used most often when working with Git day to day.

Command	What It Does
`git init`	Turns the current folder into a new, empty Git repository by creating a hidden .git folder that will track all future changes.
`git clone <url>`	Copies an existing repository from GitHub (or any remote) onto your own computer, including its full history.
`git status`	Shows which files have been changed, which are staged for the next commit, and which are still untracked.
`git add <file>`	Moves a changed file into the staging area, marking it as ready to be included in the next commit. Use git add . to stage everything at once.
`git commit -m "message"`	Saves the staged changes as a permanent snapshot in the project history, along with a short message describing what was done.
`git push`	Uploads your local commits to the remote repository on GitHub so others (and your future self) can see them.
`git pull`	Downloads the latest commits from GitHub and merges them into your local branch, keeping your copy up to date.
`git branch`	Lists all branches in the repository, or creates a new one if a name is given (e.g. git branch new-feature).
`git checkout / switch`	Moves you to a different branch. git switch is the newer, simpler command introduced to replace this use of checkout.
`git merge <branch>`	Combines the changes from another branch into the branch you are currently on, usually after a feature is finished.

```
MINGW64:/c/Users/Umama/projects/portfolio

$ git init
Initialized empty Git repository in /c/Users/Umama/projects/portfolio/.git/
$ git status
On branch main
No commits yet
Untracked files: index.html, style.css
$ git add .
$ git commit -m "Initial commit"
[main (root-commit) 3a1f9c2] Initial commit
 2 files changed, 84 insertions(+)
$ git remote add origin https://github.com/Umama2007/portfolio.git
$ git push -u origin main
Enumerating objects: 4, done.
Writing objects: 100% (4/4), done.
branch 'main' set up to track 'origin/main'.
$ git branch feature-navbar
$ git checkout feature-navbar
Switched to branch 'feature-navbar'
$
```

Figure 7: A Git Bash session running a typical sequence of commands, from git init through to git push.

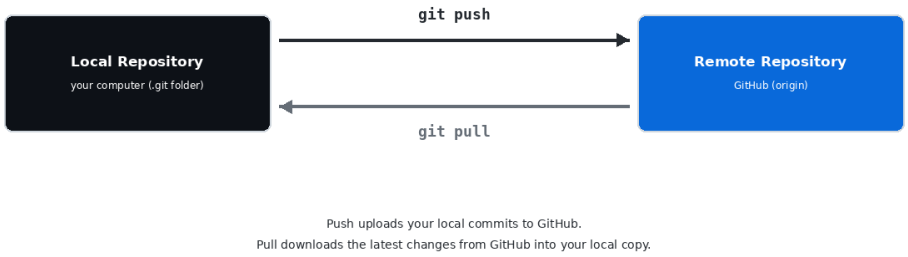

Figure 8: How git push and git pull move commits between a local and a remote repository.

Pros and Cons

Advantages of Git

- Works fully offline, since the entire history is stored locally.
- Extremely fast for common operations such as committing and branching.
- Free, open-source, and supported on every major operating system.

Advantages of GitHub

- Makes remote collaboration and code review straightforward through pull requests.
- Free hosting for public repositories, which is ideal for students and portfolios.
- Built-in automation through GitHub Actions removes a lot of manual testing and deployment work.

Disadvantages

- Git's command-line interface has a real learning curve and can feel intimidating to beginners.
- Merge conflicts, though normal, can be confusing to resolve without practice.
- GitHub requires an internet connection, and private repositories on some older plans came with limits on collaborators.

Conclusion

Git and GitHub solve two related but different problems: Git keeps track of a project's history on a local machine, while GitHub gives that history a shared home in the cloud along with tools for collaboration. Together, they have become close to essential for modern software development, whether the project is a solo assignment, a freelance client build, or an open-source effort involving hundreds of contributors. For a student moving into software or AI development, learning to use both comfortably is less of an optional skill and more of a basic requirement for working the way the rest of the industry already does.

Umama Ume Amen umamasajid25@gmail.com